

NUCLEO-G491RE üzerinde 3 fazlı BLDC komutasyon simülasyonu

Neyi, neden, hangi sırayla yaptığımı adım adım anlatan tekrar notu.

CubeMX → CubeIDE → ST-LINK → CoolTerm zinciri.

PROJE	MCU	SERİ HAT	HIZ
staj_motor_simule_g491	STM32G491RET6	LPUART1 / VCP	115200 8-N-1

İÇİNDEKİLER

1. Amaç ve güvenlik sınırı
2. Neden Nucleo?
3. CubeMX: proje kurulumu
4. CubeIDE: kodun mantığı
5. Derleme ve karta yazma
6. CoolTerm: metin çıktısı
7. CoolTerm Chart: grafik
8. Kontrol listesi
9. Sonraki adımlar
10. Terimler sözlüğü
11. Kendi izleme uygulamamız: CoolTerm'ün ötesi
12. Ek - main.c satır satır

01 Amaç ve güvenlik sınırı

Bu uygulama gerçek bir BLDC motoru döndürmez. Üç GPIO pini üzerinde BLDC'nin altı adımlı (six-step) komutasyon sırasını canlandırır. Her 200 ms sonunda A, B ve C fazının durumu değişir; aynı bilgi LPUART1 ile bilgisayara gönderilir ve CoolTerm'de metin veya grafik olarak izlenir.

? **Neden gerçek motor değil?** PA4, PB4 ve PB5 yalnızca 3.3 V mantık çıkışıdır. Motor fazı veya MOSFET kapısı bu pinlere doğrudan bağlanmaz. Gerçek sistemde 3-faz inverter, gate driver, akım ölçümü ve PWM gerekir. Bu çalışma yalnızca komutasyon mantığını güvenli biçimde öğretir.

```
Komutasyon tablosu → GPIO A/B/C → LPUART1 TX  
→ ST-LINK Virtual COM Port → CoolTerm
```

02 Neden Nucleo kartı işi kolaylaştırır?

Kart üzerindeki ST-LINK V3E iki ayrı iş yapar: **SWD** ile firmware yazma ve hata ayıklama, **Virtual COM Port (VCP)** ile UART verisini USB üzerinden Mac'e taşıma. Bu yüzden ek USB-UART dönüştürücü ve DFU modu gerekmez.

Mac + CoolTerm

| micro-USB veri kablosu



NUCLEO üzerindeki ST-LINK V3E

| SWD ◀▶ PA13 (SWDIO), PA14 (SWCLK) : programlama / debug

| VCP ◀▶ PA2 (LPUART1_TX), PA3 (LPUART1_RX) : UART



STM32G491RET6

? **Neden Board Selector ile kart seçtik, sadece MCU değil?** CubeMX kart şablonunda ST-LINK ve VCP bağlantılarını (PA2/PA3) tanır. Yalnız MCU seçilirse bu fiziksel bağlantıları kendin doğrulaman gerekir.

03 STM32CubeMX ile sıfırdan proje

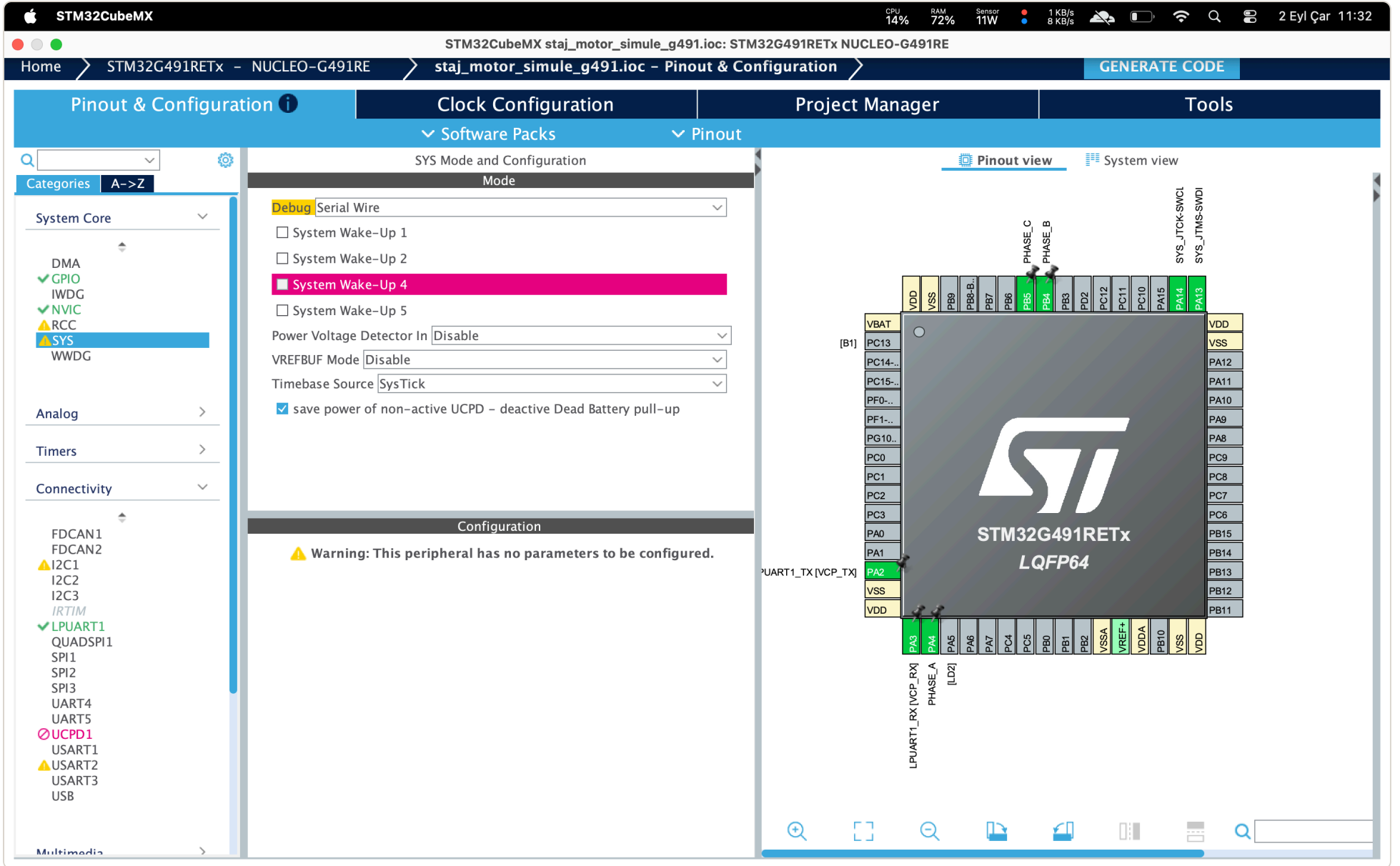
3.1 Kart seçimi

1. STM32CubeMX'i aç, **Access to Board Selector** seç.
2. Arama alanına NUCLEO-G491RE yaz.
3. Kartı seç, **Start Project**.

? **LPUART alanı gri veya kilitli görünürse?** Kart şablonu bazı pinleri önceden ayırmış olabilir. Pinout menüsünden Clear Pinout / Reset Configuration ile temizleyip ayarları yeniden yap. Mevcut projede rastgele yapma; önceki pin ayarlarını siler.

3.2 SYS: Serial Wire debug

System Core > **SYS** ekranında Debug = Serial Wire seçilir. Timebase Source SysTick olarak kalır.



Şekil 1 · SYS > Debug = Serial Wire. Sağdaki pinout'ta PA13/PA14 SWD için ayrılmış (yeşil).

? **Neden Serial Wire?** SWDIO (PA13) ve SWCLK (PA14) pinlerini ST-LINK programlayıcısına ayırır. "No Debug" seçilirse tekrar programlama gereksiz derecede zorlaşır. Bu iki pini başka GPIO için kullanma.

3.3 LPUART1: bilgisayara seri veri

Connectivity > **LPUART1** ekranında:

Alan	Değer
Mode	Asynchronous
Baud Rate	115200
Word Length / Parity / Stop	8 Bits / None / 1
Hardware Flow Control	Disable
Pinler	PA2 = LPUART1_TX, PA3 = LPUART1_RX

? **Neden 115200 8-N-1?** Terminal uygulamalarında yaygın ve bu proje için yeterince hızlı. 8 veri biti, parity yok, 1 stop biti. CoolTerm tarafında da aynısı seçilmelidir. CTS/RTS kullanmadığımız için flow control kapalı.

? **Henüz komut almıyorsak RX'i neden ayarladık?** TX, STM32'nin gönderdiği hat; STEP satırları PA2'den çıkar. PA3 (RX) şimdilik boş, ama ileride bilgisayardan başlat/durdur/hız komutu göndermek için altyapı hazır olur.

3.4 Üç faz GPIO çıkışı

Pin	User Label	GPIO ayarı
PA4	PHASE_A	Output Push Pull, Low, No pull, Low speed
PB4	PHASE_B	Output Push Pull, Low, No pull, Low speed
PB5	PHASE_C	Output Push Pull, Low, No pull, Low speed

? **Neden bu üç pin?** Bu projede boş olan üç bağımsız GPIO. PA5 karttaki LD2 kullanıcı LED'i, o yüzden faz pini olarak seçilmedi.

? **Neden Push Pull ve başlangıç Low?** Push Pull, pin HIGH iken aktif 3.3 V, LOW iken aktif GND sürer. Başlangıcın Low olması reset sonrası güvenli durum verir. Low speed bu yavaş simülasyon için yeterli.

3.5 Clock ve kod üretimi

CubeMX'in ürettiği yapı HSI ve PLL ile 170 MHz sistem saatini kullanır. Clock Configuration ekranında elle değişiklik gerekmez.

```
HSI           = 16 MHz
PLL girişi    = 16 / 4 = 4 MHz      (PLLM = DIV4)
VCO           = 4 × 85 = 340 MHz    (PLLN = 85)
SYSCLK        = 340 / 2 = 170 MHz    (PLLR = DIV2)
```

Project Manager: Project Name `staj_motor_simule_g491`, Toolchain STM32CubeIDE, Firmware STM32Cube FW_G4, **Keep User Code** etkin. Sonra **GENERATE CODE**.

? **Keep User Code neden açık?** .ioc değişip tekrar Generate Code yapıldığında USER CODE BEGIN/END bloklarındaki uygulama kodu korunur. Kapalıysa yazdığın kod silinir.

04 STM32CubeIDE: kodun çalışma mantığı

Kod Core/Src/main.c içindedir. Eklenen kod yalnızca USER CODE bloklarına konur; CubeMX'in oluşturduğu MX_GPIO_Init() ve MX_LPUART1_UART_Init() silinmez.

4.1 Faz durumları

```
typedef uint8_t PhaseState_t;  
enum { PHASE_FLOAT = 0U, PHASE_LOW = 1U, PHASE_HIGH = 2U };
```

- **PHASE_FLOAT**: pin sürülmez, input / yüksek empedans.
- **PHASE_LOW**: pin aktif 0 V (lojik 0).
- **PHASE_HIGH**: pin aktif 3.3 V (lojik 1).

? **FLOAT ile LOW aynı şey değil mi?** Değil. LOW aktif olarak GND'ye çeker; FLOAT hiç sürmez. Gerçek motor sürücüsünde FLOAT, ilgili faz kolunun tamamen kapatılmasına (her iki MOSFET off) karşılık gelir.

4.2 Altı adımlı komutasyon tablosu

Adım	PHASE_A	PHASE_B	PHASE_C
1	HIGH (+1)	LOW (-1)	FLOAT (0)
2	HIGH (+1)	FLOAT (0)	LOW (-1)
3	FLOAT (0)	HIGH (+1)	LOW (-1)
4	LOW (-1)	HIGH (+1)	FLOAT (0)
5	LOW (-1)	FLOAT (0)	HIGH (+1)
6	FLOAT (0)	LOW (-1)	HIGH (+1)

```
#define MOTOR_STEP_COUNT      6U
#define MOTOR_STEP_PERIOD_MS  200U

static const uint8_t commutation_table[MOTOR_STEP_COUNT] = {
    PACK_PHASES(PHASE_HIGH,  PHASE_LOW,  PHASE_FLOAT),
    PACK_PHASES(PHASE_HIGH,  PHASE_FLOAT, PHASE_LOW),
    PACK_PHASES(PHASE_FLOAT, PHASE_HIGH,  PHASE_LOW),
    PACK_PHASES(PHASE_LOW,   PHASE_HIGH,  PHASE_FLOAT),
    PACK_PHASES(PHASE_LOW,   PHASE_FLOAT, PHASE_HIGH),
    PACK_PHASES(PHASE_FLOAT, PHASE_LOW,   PHASE_HIGH)
};
```

? **Neden her adımda bir HIGH, bir LOW, bir FLOAT?** Six-step komutasyonda akım her an yalnızca iki sargıdan geçer; üçüncü sargı serbest bırakılır (gerçek motorda back-EMF ölçümü buradan yapılır). Altıncı adımdan sonra tekrar birinciye dönülür.

? **PACK_PHASES ne işe yarıyor?** Üç faz durumunu ikişer bit halinde tek byte'ta saklar (3 durum için 2 bit yeter). UNPACK_PHASE seçilen adımdan A/B/C bilgisini geri ayırır. Tablo küçük ve okunur kalır.

4.3 Set_Phase: HIGH, LOW ve FLOAT uygulanması

```
static void Set_Phase(GPIO_TypeDef *port, uint16_t pin, PhaseState_t state)
{
    GPIO_InitTypeDef gpio = {0};
    gpio.Pin    = pin;
    gpio.Pull   = GPIO_NOPULL;
    gpio.Speed  = GPIO_SPEED_FREQ_LOW;

    if (state == PHASE_FLOAT) {
        gpio.Mode = GPIO_MODE_INPUT;           // sürme yok
        HAL_GPIO_Init(port, &gpio);
    } else {
        gpio.Mode = GPIO_MODE_OUTPUT_PP;
        HAL_GPIO_Init(port, &gpio);
        HAL_GPIO_WritePin(port, pin,
            (state == PHASE_HIGH) ? GPIO_PIN_SET : GPIO_PIN_RESET);
    }
}
```

? **Neden her çağrıda HAL_GPIO_Init tekrar çağrılıyor?** FLOAT için pinin modunu Input'a, HIGH/LOW için Output'a çevirmek gerekiyor. Sadece WritePin ile FLOAT üretilemez; mod değişimi şart.

4.4 Motor_Advance, SysTick ve UART

Motor_Advance() güncel adımı seçer, A/B/C pinlerini günceller ve aynı bilgiyi LPUART1 üzerinden yollar. Ana döngüde zamanlama:

```
const uint32_t now_ms = HAL_GetTick();
if ((uint32_t)(now_ms - last_step_ms) >= MOTOR_STEP_PERIOD_MS)
{
    last_step_ms += MOTOR_STEP_PERIOD_MS;
    Motor_Advance();
}
```

STEP 1		A: 1		B: -1		C: 0
STEP 2		A: 1		B: 0		C: -1
STEP 3		A: 0		B: 1		C: -1
STEP 4		A: -1		B: 1		C: 0
STEP 5		A: -1		B: 0		C: 1
STEP 6		A: 0		B: -1		C: 1

? **Neden HAL_Delay(200) değil?** HAL_Delay işlemciyi bloklar. HAL_GetTick ile fark almak ana döngüyü serbest bırakır; ileride buton okuma veya UART RX eklemek kolaylaşır. last_step_ms += yazımı da birikimli kaymayı önler.

? **Tam bir tur ne kadar sürer?** $6 \times 200 \text{ ms} = 1200 \text{ ms}$, yani 1.2 s.

05 Derleme, karta yazma ve debug

CubeIDE'de proje adına tıkla ve çekiç simgesine bas. Beklenen çıktı:

```
11:12:12 Build Finished. 0 errors, 0 warnings. (took 934ms)
→ Debug/staj_motor_simule_g491.elf
```

Kart USB ile bağlıyken Run/Debug ile yükleme yapılır. ST-LINK firmware'i SWD üzerinden yazar ve MCU'yu resetler. DFU moduna gerek yok.

? **"ST-Link Server is required to launch the debug session" uyarısı?** Kod hatası değil; macOS'ta ST-LINK Server bileşeni eksik. Kurup IDE'yi yeniden aç, ya da STM32CubeProgrammer'da ST-LINK/SWD ile .elf dosyasını programla.

06 CoolTerm ile UART verisini görme

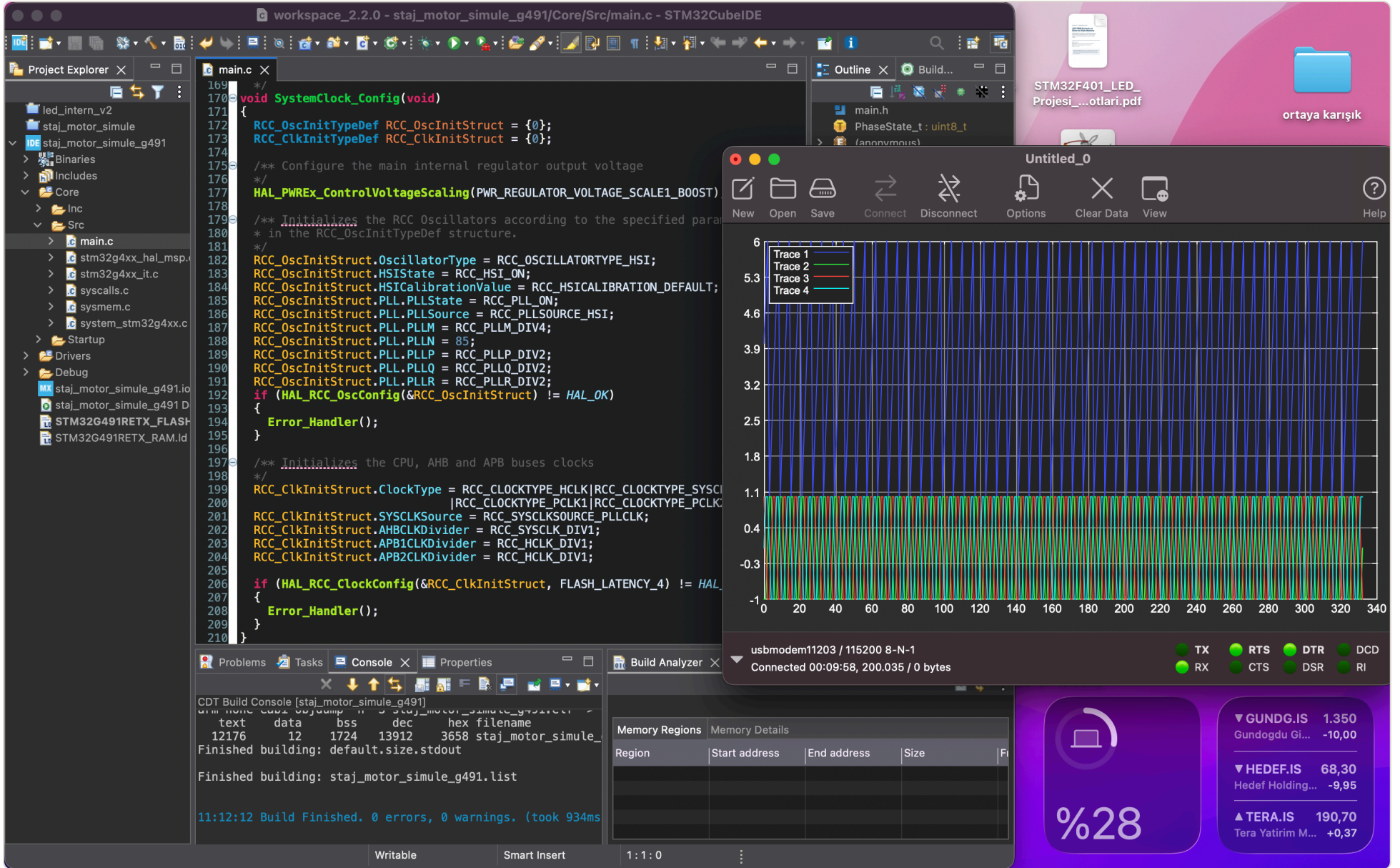
Alan	Değer
Serial Port	/dev/cu.usbmodem...
Baudrate	115200
Data / Parity / Stop	8 / None / 1
Flow Control	None

Connect'e basınca STEP 1–6 satırları yaklaşık her 200 ms'de bir görünür (Şekil 4, sağ panel).

? **Port adı neden değişiyor?** Sondaki numara her bağlantıda farklı olabilir; önemli olan NUCLEO/ST-LINK'e ait `/dev/cu.usbmodem...` girdisini seçmek.

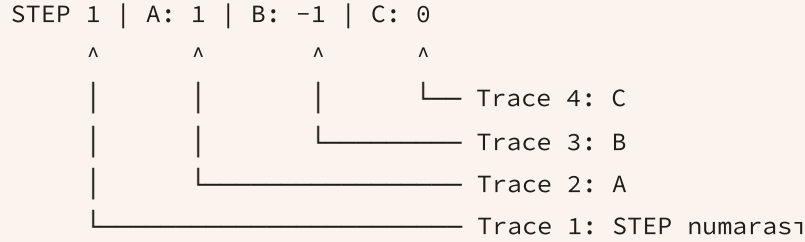
? **Metin bozuk geliyorsa?** İlk kontrol baud: hem CubeMX hem CoolTerm 115200. Sonra 8-N-1 ve Flow Control=None. Port görünmüyorsa veri taşıyan USB kablosu kullan; başka bir terminalin portu açık tutmadığından emin ol.

07 CoolTerm Chart: ayar ve yorum



Şekil 5 · Ham chart: Trace 1 (mavi) 1–6 arasında yükselen STEP numarası; Trace 2–4 ise –1/0/+1 arasında basamaklanan A/B/C fazları.

7.1 Neden dört trace oluşur?



CoolTerm satırdaki sayıları sırayla ayıklar. 0–6 arasında yükselen ilk eğri faz gerilimi değil, adım numarasıdır.

7.2 Temiz grafik için ayar

1. **View > Chart** görünümüne geç.
2. Grafik alanında sağ tık → **Chart Properties**.
3. *Only plot lines with exactly the specified number of data points* aç, değeri **4** yap.
4. Sağ tık → **Trace Properties**; Trace 1 için Visible kapat.
5. Trace 2: Phase A, Trace 3: Phase B, Trace 4: Phase C olarak adlandır.
6. **Clear Data**; 1–2 s yeni veri biriktir. Zoom: View > Zoom Chart veya Control + scroll.

7.3 Grafiği yorumlama

Y değeri	Anlamı
+1	Faz GPIO'su aktif 3.3 V (HIGH) sürüyor.
0	FLOAT; pin input / yüksek empedans, aktif sürme yok.
-1	Faz GPIO'su aktif 0 V / GND (LOW) sürüyor.

A: +1, +1, 0, -1, -1, 0, ...

B: -1, 0, +1, +1, 0, -1, ...

C: 0, -1, -1, 0, +1, +1, ...

? **Eğriler neden sinüs değil?** Six-step komutasyon basamaklı/trapezoidal sinyal üretir. Her adımda bir faz HIGH, bir faz LOW, bir faz FLOAT görülmesi doğru sonuçtur. Fazlar birbirine 120° (2 adım) kaymıştır.

? **Hem okunaklı metin hem temiz chart istersem?** İki satır türü gönder: STEP 1 | A: 1 | B: -1 | C: 0 terminal için, 1,-1,0 chart için. Chart Properties'te "yalnızca 3 sayı içeren satırları çiz" filtresi seçilir.

08 Başarı kontrol listesi

- ☐ **CubeIDE** · 0 errors, 0 warnings
- ☐ **ST-LINK** · Firmware karta yazılabiliyor
- ☐ **CoolTerm** · /dev/cu.usbmodem..., 115200 8-N-1, Flow Control=None
- ☐ **Terminal** · STEP 1'den 6'ya sıra sürekli tekrar ediyor
- ☐ **Chart** · A/B/C yalnızca -1, 0, +1 seviyelerinde

09 Sonraki adımlar

1. PA5/LD2 LED'ini her komutasyon döngüsünde değiştir.
2. PC13/B1 butonuyla başlat/durdur ekle.
3. UART RX ile s, p, +, - komutlarını al.
4. HAL_GetTick() yerine timer interrupt kullan.
5. GPIO yerine timer PWM kanallarını öğren.
6. Hall sensörü / back-EMF geri beslemesi ve gerçek gate driver ekle.

Özet • CubeMX donanımı tanımlar; CubeIDE HAL kodunu derler; ST-LINK firmware'i SWD ile karta yazar; LPUART1 veriyi ST-LINK VCP üzerinden Mac'e taşır; CoolTerm veriyi terminal ve grafik olarak görünür yapar.

10 Terimler sözlüğü

BLDC	Fırçasız DC motor. Komütasyon mekanik fırça yerine elektronik olarak yapılır.
Six-step	Altı adımlı trapezoidal komutasyon; her adımda iki faz sürülür, biri serbest.
HAL	Hardware Abstraction Layer. ST'nin çevre birimlerini soyutlayan C kütüphanesi.
GPIO	Genel amaçlı giriş/çıkış pini.
Push Pull	Pinin hem 3.3 V'a hem GND'ye aktif sürebildiği çıkış modu.
Yüksek empedans	Pinin sürülmediği, "boşta" durum (FLOAT / input modu).
LPUART	Low-Power UART. Asenkron seri haberleşme birimi.
8-N-1	8 veri biti, parity yok, 1 stop biti.
SWD	Serial Wire Debug. İki telli (SWDIO, SWCLK) programlama/debug arayüzü.
VCP	Virtual COM Port. ST-LINK'in UART'ı USB üzerinden seri port gibi göstermesi.
HSI / PLL	Dahili 16 MHz osilatör ve onu 170 MHz'e çarpan faz kilitli döngü.
SysTick	Cortex-M çekirdek zamanlayıcısı; HAL_GetTick() ms sayacını besler.
.ioc	CubeMX proje dosyası; pin ve çevre birimi ayarlarını tutar.
.elf	Derlenmiş firmware dosyası; ST-LINK bunu karta yazar.

11 Kendi izleme uygulamamız: CoolTerm'ün ötesi

CoolTerm genel amaçlı bir terminal; satırdaki sayıları çizer ama STEP'in ne olduğunu bilmez. Motor kontrolüne özel bir uygulama, gelen veriyi *anlayan* ve karta komut da gönderebilen bir araç olur. Bu bölüm, şu anki step takibinden ileride gerçek motor kontrolüne kadar neyin eklenebileceğini planlar.

? **CoolTerm varken neden kendi uygulamamızı yazalım?** Üç eksik var: (1) Trace 1'in STEP olduğunu bilmiyor, hep elle filtre gerekiyor; (2) tek yönlü, karta hız/başlat/durdur gönderemiyor; (3) veri anlamsız sayı; RPM, adım süresi, hata sayısı gibi türetilmiş bilgi üretmiyor.

11.1 Mimari: veri zinciri değişmiyor, sadece son halka

KART TARAFI

MAC · IZLEME UYGULAMASI



Şekil 6 · Uygulama mimarisi. Sol sütun mevcut zincirin aynısı; sağdaki üç katman yeni yazılacak kısım. Katmanlar arasında yalnızca ok yönündeki bağımlılık var.

Uygulama üç katmandan oluşur: **seri katman** (port aç/kapat, byte oku/yaz), **veri katmanı** (satırı ayrıştır, tampona yaz, istatistik üret), **görünüm katmanı** (terminal, grafik, kontrol paneli). Katmanlar ayrı tutulursa ileride protokol değişince yalnızca parser değişir.

11.2 Teknoloji seçenekleri

Yol	Seri erişim	Grafik	Ne zaman uygun
Python	pyserial	pyqtgraph / matplotlib	En hızlı başlangıç; staj süresinde bitirilebilir. PyQt ile masaüstü pencere.
Web (tarayıcı)	Web Serial API	Canvas / uPlot	Kurulum yok, Chrome'da çalışır. macOS'ta doğrudan usbmodem'e bağlanır.
Masaüstü (Electron / Tauri)	serialport (Node) / Rust	Web grafik kütüphaneleri	Dağıtılabılır .app istenirse. Web sürümünün üstüne inşa edilir.
C++ / Qt	QSerialPort	QtCharts	Endüstriyel araç hedefleniyorsa; öğrenme eğrisi en dik.

? **Hangisiyle başlamalı?** Python + pyserial + pyqtgraph. Seri okuma 10 satır, canlı grafik 20 satır kod. Protokol oturduktan sonra web veya masaüstüne taşınır; parser mantığı aynı kalır.

11.3 Aşama 1 · Şimdiki firmware ile yapılabilenler

Firmware değişmeden, mevcut STEP n | A: x | B: y | C: z satırıyla:

Terminal

Ham satırlar, zaman damgalı. Renk: HIGH turuncu, LOW mavi, FLOAT gri.

Faz grafiđi

A/B/C ayrı eksenlerde, -1/0/+1 basamaklı. STEP otomatik olarak grafikten ayrılır.

Komutasyon göstergesi

6 dilimli daire; aktif adım vurgulu. Fazların motor etrafında dönüşü görünür olur.

Türetilmiş değerler

Adım süresi (ms), elektriksel RPM = $60000 / (6 \times \text{periyot})$, toplam tur, kaçırılan adım sayısı.

Doğrulama

Her satırda tam bir HIGH, bir LOW, bir FLOAT var mı? Adım sırası 1→6 mı? Hata sayacı.

Kayıt

CSV/JSON'a zaman damgalı log; sonra offline analiz veya tekrar oynatma.

? **Adım süresi neden ölçülmeli?** Firmware 200 ms diyor; bilgisayar tarafında ölçülen süre farklıysa ya USB tamponlaması ya da ana döngüde bloklama var. Bu ölçüm ileride timer interrupt'a geçişin etkisini de gösterir.

11.4 Aşama 2 · İki yönlü: karta komut göndermek

Firmware'de UART RX açılır (PA3 zaten ayarlı). Basit tek karakterlik komut seti:

Komut	Anlamı	Uygulamada
s	Start	Başlat düğmesi
p	Pause / stop	Durdur düğmesi
+ / -	Periyodu ± 50 ms değiştir	Hız kaydırıcısı
d	Yön değiştir (tabloyu tersten gez)	CW / CCW anahtarı
n	Tek adım ilerle (manuel mod)	Adım düğmesi
?	Durum raporu iste	Bağlanınca otomatik

? **Firmware tarafında RX nasıl alınır?** HAL_UART_Receive_IT(&hlpuart1, &rx_byte, 1) ile interrupt bazlı tek byte; CubeMX'te LPUART1 global interrupt kutusu işaretlenir (Şekil 2'de NVIC Settings). Ana döngü bloklanmaz, komut anında işlenir.

11.5 Protokolü büyütme: metinden yapılandırılmış veriye

Şu anki satır insan için okunaklı ama makine için gevşek. Uygulama büyüdükçe seçenekler:

```
# A • CSV satırı (basit, CoolTerm ile de uyumlu)
```

```
t_ms,step,a,b,c,period_ms
```

```
12400,3,0,1,-1,200
```

```
# B • JSON satırı (alan eklemek kolay, kendi uygulaman için)
```

```
{"t":12400,"step":3,"ph":[0,1,-1],"per":200,"dir":1}
```

```
# C • Binary çerçeve (yüksek hız, gerçek motor için)
```

```
[0xAA][len][type][payload...][crc8]
```

? **Neden zaman damgasını kart göndersin?** Bilgisayarın okuma zamanı USB tamponu yüzünden ± 10 ms oynar. HAL_GetTick() değeri satıra eklenirse grafik gerçek zamanı gösterir; periyot analizi doğru olur.

? **Binary'ye ne zaman geçilir?** PWM ve akım ölçümü eklendiğinde ms başına onlarca örnek gelir; metin 115200 baud'u doldurur. Binary çerçeve + CRC hem hız hem hata tespiti sağlar. O noktada baud da 921600'e çıkarılabilir.

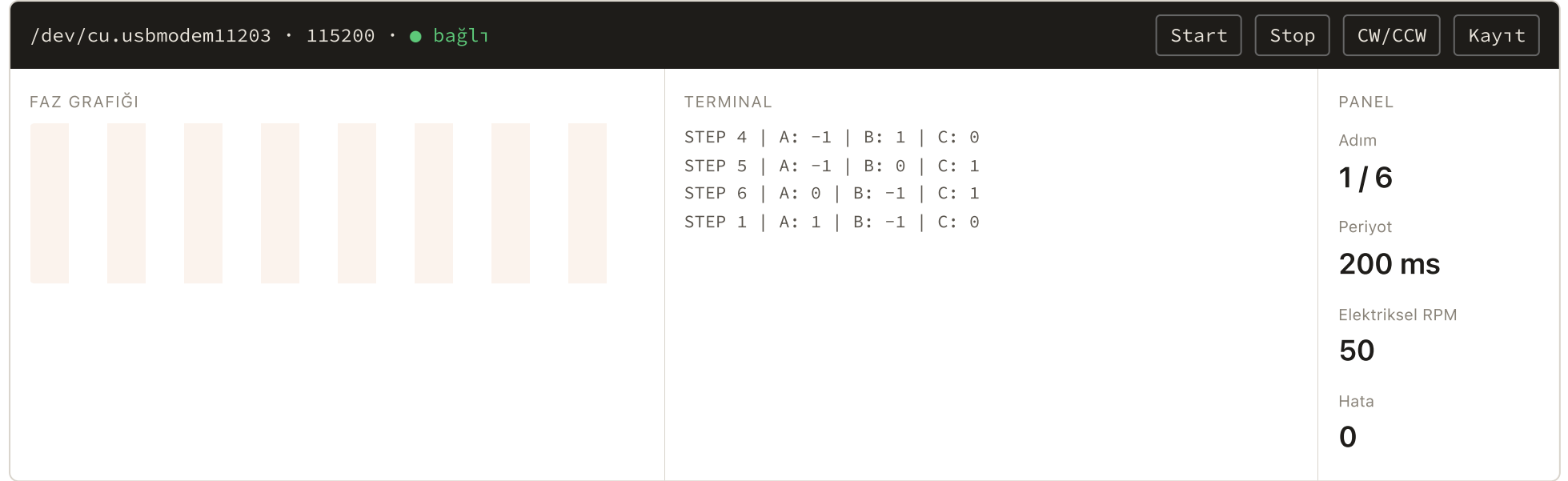
11.6 Aşama 3 • Gerçek motor kontrolüne doğru izlenecekler

Firmware geliştikçe (bkz. Bölüm 9) uygulamaya eklenecek paneller:

Firmware adımı	Karttan gelen yeni veri	Uygulamada görünüm
Timer interrupt	Kesin periyot, jitter	Periyot histogramı; hedef vs ölçülen
Timer PWM	Duty cycle (%), PWM frekansı	Duty kaydırıcısı; faz grafiği -1/0/+1 yerine -duty...+duty
Hall sensörü	Hall durumu (3 bit), gerçek rotor konumu	Komutasyon dairesinde beklenen vs ölçülen adım; senkron hatası
Back-EMF	FLOAT fazın ADC gerilimi, sıfır geçiş anı	Osiloskop görünümü; sıfır geçiş işaretleri
Akım ölçümü	Faz akımları, bus akımı	Akım grafiği; aşırı akım alarmı ve eşik çizgisi
Kapalı çevrim hız	Hedef RPM, ölçülen RPM, PID çıkışı	Hedef/ölçülen üst üste; PID kazançlarını canlı ayarlama
Koruma	Hata kodları, sıcaklık, bus gerilimi	Durum ışıkları; hata günlüğü; acil durdur düğmesi

? **Gerçek motorda acil durdur nasıl güvenli olur?** Yazılım komutuna tek başına güvenilmez; USB kopabilir. Uygulama p gönderir ama firmware'de ayrıca watchdog olmalı: belirli süre komut gelmezse fazları FLOAT'a al. Donanım koruması (sigorta, akım limiti) her zaman yazılımın üstündedir.

11.7 Ekran taslağı



Şekil 7 · Üstte bağlantı ve komut çubuğu; solda faz grafiği, ortada terminal, sağda türetilmiş değerler paneli. Komutasyon dairesi ikinci sekmede yer alabilir.

11.8 Geliştirme sırası

1. Port listele, bağlan, ham satırları göster. CoolTerm ile birebir aynı davranış: referans noktası.
2. Parser: STEP n | A: x | B: y | C: z → {step, a, b, c, t}. Hatalı satırları say, atma.
3. Faz grafiği + komutasyon dairesi. Aşama 1 türetilmiş değerler.
4. CSV kayıt ve tekrar oynatma. Kartsız geliştirme mümkün olur.
5. Firmware'e RX ekle; s / p / + / - komutları. Hız kaydırıcısı.
6. Protokolü CSV veya JSON'a taşı; zaman damgasını karta al.
7. Firmware ilerledikçe 11.6'daki panelleri tek tek ekle.

? **Kayıt/tekrar oynatma neden erken?** Bir kez kaydedilen 30 saniyelik log ile grafik ve panel kartsız geliştirilir; her denemede kartı bağlamak gerekmez. Hata ayıklamada "o an ne oldu" sorusunu da cevaplar.

12 Ek · main.c satır satır

Bölüm 4 mantığı anlattı; burada dosyanın tamamı yukarıdan aşağıya, her blok yanında ne yaptığı ile. CubeMX'in ürettiği kısımlar gri, bizim eklediğimiz USER CODE blokları koyu arka planda.

12.1 Dosyanın haritası

#include "main.h"	HAL, GPIO ve UART tanımları. CubeMX üretir.
USER CODE PTD / PD / PM	Bizim: PhaseState_t, sabitler, PACK/UNPACK makroları.
hlpuart1	UART tanıtıcısı (handle). CubeMX üretir.
USER CODE PV	Bizim: current_step, commutation_table, chart metinleri.
Prototipler	SystemClock_Config, MX_GPIO_Init, MX_LPUART1_UART_Init. CubeMX üretir.
USER CODE PFP + 0	Bizim: Set_Phase ve Motor_Advance prototipleri ve gövdeleri.
main()	HAL_Init → SystemClock_Config → MX_GPIO_Init → MX_LPUART1_UART_Init → USER CODE 2 → while(1) { USER CODE 3 }
SystemClock_Config	HSI + PLL → 170 MHz. CubeMX üretir (Şekil 4'te görünen kod).
MX_LPUART1_UART_Init	115200 8-N-1, TX/RX. CubeMX üretir.
MX_GPIO_Init	Port saatlerini açar, PA4/PB4/PB5'i Output PP Low yapar. CubeMX üretir.
Error_Handler	Kesmeleri kapatır, sonsuz döngü. CubeMX üretir.

? **USER CODE bloklarının adları ne anlama geliyor?** PTD = Private TypeDef, PD = Private Define, PM = Private Macro, PV = Private Variables, PFP = Private Function Prototypes, 0 = fonksiyon gövdeleri, 2 = main içinde init sonrası, 3 = while döngüsü içi. Bu sıra CubeMX'in kendi dosya düzenidir.

12.2 Tipler, sabitler, makrolar

```
/* USER CODE BEGIN PTD */
typedef uint8_t PhaseState_t;
enum { PHASE_FLOAT = 0U, PHASE_LOW = 1U, PHASE_HIGH = 2U };
/* USER CODE END PTD */

/* USER CODE BEGIN PD */
#define MOTOR_STEP_COUNT      6U
#define MOTOR_STEP_PERIOD_MS 200U
/* USER CODE END PD */

/* USER CODE BEGIN PM */
#define PACK_PHASES(a, b, c)  (uint8_t)((((a) << 4) | ((b) << 2) | (c)))
#define UNPACK_PHASE(v, shift) (PhaseState_t)((((v) >> (shift)) & 0x03U))
/* USER CODE END PM */
```

Satır	Ne yapar
<code>typedef uint8_t PhaseState_t</code>	Faz durumunu 1 byte'lık isimli tip yapar; fonksiyon imzaları okunur olur.
<code>enum { FLOAT=0, LOW=1, HIGH=2 }</code>	Üç durum, 2 bite sığar (0–3). FLOAT'ın 0 olması: sıfırlanmış bir değer güvenli durumdur.
<code>MOTOR_STEP_PERIOD_MS 200U</code>	Adım süresi. Tek yerden değişir; ileride + / – komutu bunu değiştirecek bir değişkene dönüşür.
<code>PACK_PHASES(a,b,c)</code>	A'yı 4–5. bite, B'yi 2–3. bite, C'yi 0–1. bite koyar: 00AA BBCC. Tablo 6 byte tutar.
<code>UNPACK_PHASE(v, shift)</code>	Sağa kaydır, & 0x03 ile 2 biti al. shift: A için 4, B için 2, C için 0.
<code>U soneki</code>	Sabit unsigned yapar; işaretli/işaretsiz karşılaştırma uyarılarını önler. "0 warnings" bu yüzden.

12.3 Değişkenler

```
/* USER CODE BEGIN PV */
static uint8_t current_step = 0U;

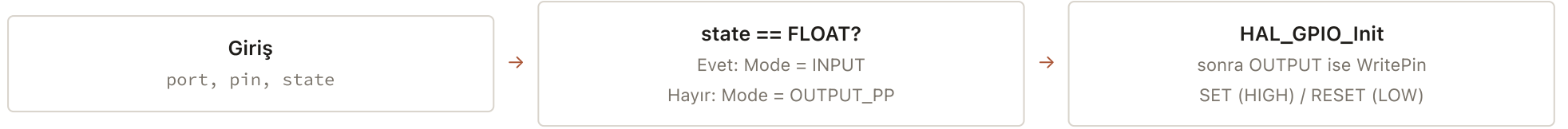
static const uint8_t commutation_table[MOTOR_STEP_COUNT] = {
    PACK_PHASES(PHASE_HIGH, PHASE_LOW, PHASE_FLOAT), /* 1 */
    PACK_PHASES(PHASE_HIGH, PHASE_FLOAT, PHASE_LOW), /* 2 */
    PACK_PHASES(PHASE_FLOAT, PHASE_HIGH, PHASE_LOW), /* 3 */
    PACK_PHASES(PHASE_LOW, PHASE_HIGH, PHASE_FLOAT), /* 4 */
    PACK_PHASES(PHASE_LOW, PHASE_FLOAT, PHASE_HIGH), /* 5 */
    PACK_PHASES(PHASE_FLOAT, PHASE_LOW, PHASE_HIGH) /* 6 */
};

static const char chart_lines[][40] = { ... }; /* CoolTerm başlığı */
static const char chart_legend[] = "...";
/* USER CODE END PV */
```

Satır	Ne yapar
<code>static uint8_t current_step</code>	0–5 arası indeks. static: yalnızca bu dosyadan görünür, değer çağrılar arasında korunur.
<code>static const uint8_t table[]</code>	const ile Flash'a yerleşir, RAM harcamaz. Build çıktısındaki <code>data = 12 byte</code> 'ın küçük kalmasının nedeni.
<code>chart_lines / chart_legend</code>	Bağlantı anında bir kez gönderilen açıklama satırları; CoolTerm'de veri başlamadan önce ne izlendiğini gösterir.

12.4 Set_Phase

Kod Bölüm 4.3'te. Burada yalnızca akış:



? **GPIO_InitTypeDef gpio = {0} neden?** Yapının tüm alanlarını sıfırlar; Alternate gibi kullanmadığımız alanlar rastgele değer taşımaz. Yığında (stack) tanımlanan yapılar sıfırlanmadan gelir.

12.5 Motor_Advance

```
static void Motor_Advance(void)
{
    const uint8_t packed = commutation_table[current_step];
    const PhaseState_t a = UNPACK_PHASE(packed, 4);
    const PhaseState_t b = UNPACK_PHASE(packed, 2);
    const PhaseState_t c = UNPACK_PHASE(packed, 0);

    Set_Phase(PHASE_A_GPIO_Port, PHASE_A_Pin, a);
    Set_Phase(PHASE_B_GPIO_Port, PHASE_B_Pin, b);
    Set_Phase(PHASE_C_GPIO_Port, PHASE_C_Pin, c);

    char line[48];
    const int n = snprintf(line, sizeof line, "STEP %u | A: %d | B: %d | C: %d\r\n",
                          (unsigned)(current_step + 1U),
                          Phase_To_Int(a), Phase_To_Int(b), Phase_To_Int(c));
    HAL_UART_Transmit(&hlpuart1, (uint8_t *)line, (uint16_t)n, 100U);

    current_step = (uint8_t)((current_step + 1U) % MOTOR_STEP_COUNT);
}
```

Satır	Ne yapar
<code>packed = table[current_step]</code>	Bu adımın 1 byte'ını çeker.
<code>UNPACK_PHASE(packed, 4/2/0)</code>	A, B, C'yi ayırır.
<code>PHASE_A_GPIO_Port / _Pin</code>	CubeMX'teki User Label'dan otomatik üretilen makrolar (main.h). PA4 yerine isim kullanmak pin değişirse kodu korur.
<code>Phase_To_Int</code>	HIGH→+1, LOW→-1, FLOAT→0. CoolTerm chart'ın -1/0/+1 görmesinin kaynağı.
<code>snprintf(... "\r\n")</code>	Satırı tamponda kurar. \r\n terminalin satır atlaması için; CoolTerm satır sonunu buradan tanır.
<code>HAL_UART_Transmit(..., 100U)</code>	Bloklayan gönderim, 100 ms zaman aşımı. ~30 byte 115200'de 2.6 ms sürer; 200 ms periyoda göre ihmal edilir.
<code>(current_step + 1) % 6</code>	5'ten sonra 0'a döner; tablo döngüsel gezilir.

? **Neden önce pinler, sonra UART?** Pin güncellemesi mikrosaniyeler sürer, UART milisaniyeler. Fazlar önce değişince satır "şu an olan"ı bildirir; tersi olsaydı satır 2–3 ms ileriye anlatırdı.

12.6 main(): başlangıç ve ana döngü

```
int main(void)
{
    HAL_Init();                /* SysTick 1 ms, NVIC, Flash cache */
    SystemClock_Config();      /* 170 MHz */
    MX_GPIO_Init();            /* PA4/PB4/PB5 Output Low */
    MX_LPUART1_UART_Init();    /* 115200 8-N-1 */

    /* USER CODE BEGIN 2 */
    Send_Chart_Header();       /* chart_lines + legend → CoolTerm */
    uint32_t last_step_ms = HAL_GetTick();
    Motor_Advance();           /* beklemeden STEP 1'i uygula */
    /* USER CODE END 2 */

    while (1)
    {
        /* USER CODE BEGIN 3 */
        const uint32_t now_ms = HAL_GetTick();
        if ((uint32_t)(now_ms - last_step_ms) >= MOTOR_STEP_PERIOD_MS)
        {
            last_step_ms += MOTOR_STEP_PERIOD_MS;
            Motor_Advance();
        }
        /* USER CODE END 3 */
    }
}
```


Satır	Ne yapar
HAL_Init()	SysTick'i 1 ms'ye kurar; HAL_GetTick ve HAL_Delay buna dayanır. Her zaman ilk çağrı.
Init sırası	Saat → GPIO → UART. UART, kendi pinlerinin (PA2/PA3) GPIO saatinin açık olmasına ihtiyaç duyar; CubeMX bu sırayı korur.
İlk Motor_Advance()	Döngüye girmeden STEP 1 uygulanır; CoolTerm'e bağlanınca 200 ms boş beklenmez.
(uint32_t)(now - last)	Fark unsigned alınır: HAL_GetTick 49.7 günde taşsa bile fark doğru çıkar.
last_step_ms += PERIOD	= now_ms yazılsaydı döngünün gecikmesi her adımda birikirdi. Toplama ile ortalama periyot tam 200 ms kalır.

? **SystemClock_Config'e neden dokunmadık?** Şekil 4'teki kod CubeMX'in ürettiği hâli. 170 MHz bu iş için fazlasıyla yeterli; PLL değerlerini elle değiştirmek yanlış saat ve çalışmayan UART riski taşır. Baud hesabı bu saate göre CubeMX tarafından yapılır.

? **Build çıktısı ne diyor?** text 12176: kod + const tablolar (Flash). data 12: ilk değerli değişkenler. bss 1724: sıfırlanan RAM (hlpuart1 yapısı, yığın). Toplam 13.9 KB; G491'in 512 KB Flash'ının %3'ü.